

README / Setup Guide

Overview

This application is an employee leave management system designed to streamline the process of submitting, approving, and tracking leave requests within an organization. It provides core benefits such as improved efficiency in leave management, better tracking of leave balances, and enhanced communication between employees and managers. The target users are employees who need to request leave and managers who need to approve or reject these requests.

Key Features

- Employees can submit leave requests with details such as leave type, start date, and end date.
- Managers can approve or reject leave requests with comments.
- Employees can view their leave balances.
- The system ensures data integrity and security through validation, authentication, and authorization mechanisms.

Prerequisites

- Java 11+
- PostgreSQL 13+
- Node.js 16+
- npm 8+
- Hardware/resource requirements: Minimum 2GB RAM, 10GB storage
- Required system dependencies: Git, Maven

Project Structure

The repository layout is as follows, reflecting the architecture design:

- `backend/`: Contains the Spring Boot application with RESTful API endpoints, business logic, and data access layers.
- `frontend/`: Contains the React application for the user interface.
- `shared/`: Contains shared utilities and configurations.
- `docs/`: Contains API documentation and user guides.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd employee-leave-management`
3. Create virtual environment: `mvn clean install`
4. Install dependencies: `mvn spring-boot:run`

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- `SPRING_DATASOURCE_URL`: Controls the PostgreSQL database connection URL, format: `jdbc:postgresql://localhost:5432/leave_management`.
- `SPRING_DATASOURCE_USERNAME`: Controls the PostgreSQL username, format: `example_user`.
- `SPRING_DATASOURCE_PASSWORD`: Controls the PostgreSQL password, format: `example_password`.

Optional Environment Variables

- `JWT_SECRET` (default: `my_secret_key`): Controls the JWT secret key used for token generation.

Running the Application

Starting the Backend

1. Ensure Maven is installed and the project is built: `mvn clean install`
2. Set environment variables: `export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/leave_management`
3. Start server: `mvn spring-boot:run`
4. Verify: Backend is running on `http://localhost:8080`

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm run dev`

3. Open browser: `http://localhost:3000`
4. Verify: UI loads and connects to backend

Verification Checklist

- ☐ Backend health check responds at `GET /api/health` with 200 OK
- ☐ Frontend loads without console errors
- ☐ Basic workflow completes end-to-end (e.g., Submit and approve a leave request)
- ☐ API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change `PORT` env var or kill process on port 8080
- **Module not found:** Run `mvn clean install` again
- **Database connection error:** Verify connection string in `application.properties`, check service is running
- **Authentication error:** Verify JWT secret in `application.properties` is correct

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running
- **API connection error:** Verify backend URL in config, check CORS settings
- **Module resolution error:** Delete `node_modules` and run `npm install` again
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`

Common Workflows

Submit Leave Request

1. Navigate to the leave request form.
2. Fill in the leave details (employee ID, leave type, start date, end date).
3. Click submit.
4. Verify: Leave request is created with status "PENDING".

Approve Leave Request

1. Navigate to the leave request details.
2. Click approve and provide a comment.

3. Verify: Leave request status is updated to "APPROVED".

Deployment Notes

- Ensure environment variables are set correctly in the production environment.
- Use a secure method for managing secrets, such as environment variables or a secrets manager.
- Configure load balancers and scaling for high availability.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools
- Review error message and stack trace for clues
- Check environment variables are set correctly
- Refer to the API Documentation and User Guide for more details